

Kotlin++ Lite

Technical Implementation Report

IS2205 Mini Project

Developers

H.K.Y. Sandeepa – 24020931

K.I.U. Thisera – 24021059

Platform: Android

Technologies: Kotlin 2.2.10, Jetpack Compose, Room 2.8.4, java-diff-utils 4.15

Repository: <https://github.com/yasuru-sandeepaBO1/modern-editor>

1. Introduction

Kotlin++ Lite is a native Android text editor developed as the IS2205 mini project. The application supports Kotlin (`.kt`), Markdown (`.md`) and plain text (`.txt`) files and provides features such as file storage, syntax highlighting, Markdown preview, version comparison, version history, crash recovery and rollback.

The project was developed using a component-based approach. Each major feature was divided into a separate component and implemented in dependency order. This allowed individual parts of the system to be developed and tested before being used by later components.

This report focuses on the main implementation areas of the project:

- File storage and recovery
- Markdown preview
- Version comparison
- Data and version tracking
- Syntax highlighting
- Rollback
- Testing and verification

The main components related to these areas are Components 3, 5, 6, 7, 8, 9, 10, 11 and 12.

2. Component Structure

The project was developed using a component-based approach. The main components related to the features covered in this report are:

Component	Area
Component 3	File System
Component 5	Kotlin Syntax Engine
Component 6	Markdown Engine
Component 7	Crash Recovery
Component 8	Database / Persistence
Component 9	Diff Engine
Component 10	Incremental Delta / Patch
Component 11	Version Control
Component 12	History and Rollback

The components were implemented in dependency order. Persistence was implemented before the diff engine, the diff engine before the patch system, and the patch system before version control and rollback. This allowed each later component to build on functionality that had already

been implemented and tested.

The project also maintained separate test cases for the components. Automated tests were used for logic that could run independently of Android, while device-dependent behaviour was tested using an emulator.

3. Storage and Data Persistence

Storage was divided into three areas because the application has different requirements for the user's actual files, version history and unsaved editor data.

Storage area	Mechanism	Data stored
User files	Storage Access Framework + ContentResolver	.kt, .md and .txt files
Version history	Room / SQLite	File records, versions and deltas
Crash recovery	Private file in <code>filesDir</code>	Unsaved editor content

3.1 User File Storage

The actual Kotlin, Markdown and plain-text files are handled using the Android Storage Access Framework (SAF). The user selects a file through the Android file picker, and the application uses the returned URI to read and write the document. New File offers a `.kt / .md / .txt` type selector. Open accepts any document (`*/*`); the language is inferred from the file name, and unknown extensions are treated as plain text.

The application does not request broad storage permissions. Instead, it accesses only the documents explicitly selected by the user.

File operations are performed through `ContentResolver`. The file operations are also moved to `Dispatchers.IO` so that reading and writing files does not block the main UI thread.

When a file is selected, the application calls `takePersistableUriPermission()`. This converts the temporary URI permission obtained from the file picker into a persistent permission, allowing the selected document to be accessed again after an application restart.

When saving a file, the output stream is opened using `"wt"` mode. The `t` option truncates the existing content before the new content is written. This prevents old bytes from remaining at the end of a file when the newly saved content is shorter than the previous content.

Related component: C3 – File System.

3.2 Version History Storage

Version history is stored using Room with SQLite as the underlying database.

Three main entities are used:

- `FileEntity`
- `VersionEntity`
- `DeltaEntity`

The version and delta records are connected to their parent records through foreign keys. Cascade deletion is used so that deleting a file removes its associated versions, and deleting a version removes its associated patches.

Before saving a version, the application makes sure that the corresponding file record exists in the database. This is handled through an `ensureFileRecord()` step. This prevents a foreign-key constraint

failure when the first version of a file is saved.

Related component: C8 – Database / Persistence.

4. Crash Recovery

Unsaved editor content is handled separately from normal file storage.

A recovery file named `crash_recovery.txt` is stored in the application's private `filesDir`. A coroutine periodically writes the current editor buffer to this file.

The recovery record stores both:

- The current editor content
- The last-saved snapshot

This allows the application to determine whether the editor contained unsaved changes when the application stopped unexpectedly.

When a normal save is completed, the recovery record is deleted rather than updated. A recovery prompt is shown only when the stored content differs from the last-saved snapshot (`isDirty`). Matching content is treated as clean and is not offered for restore, even if a recovery file is present.

The recovery system is kept separate from the user's actual document because writing unsaved editor content directly to the user's file would overwrite the last successfully saved version.

4.1 Recovery Data Format

The recovery data uses a hand-written, length-prefixed format instead of a line-delimited format.

A simplified representation is:

```
RECOVERY_V1
content:///.../document/1234
untitled.kt
KOTLIN
1755412800000
180
<saved snapshot data>
204
<current content data>
```

The record contains:

- Recovery format version
- Document URI (may be empty for an unsaved new file)
- File name
- File type (KOTLIN, MARKDOWN or PLAIN_TEXT)
- Timestamp
- Saved snapshot length
- Saved snapshot data
- Current content length
- Current content data

The length of each content block is stored before the block itself. These lengths are **character counts** (`String.length`), not UTF-8 byte lengths. The file is written as UTF-8 text, but encode and decode both operate on Kotlin strings.

This is important because source code naturally contains new lines and other characters. A delimiter-based format could become ambiguous if the chosen delimiter appeared inside the source code. The length-prefixed approach allows the decoder to know exactly how many characters belong to each field.

During decoding, the recovery header is checked first and the decoding process is wrapped using `runCatching`. If the recovery data is incomplete, corrupted or invalid, the decoder returns no usable recovery record instead of allowing the recovery mechanism itself to crash the application during startup.

Related component: C7 – Crash Recovery.

5. Syntax Highlighting

Syntax highlighting was implemented separately for Kotlin and Markdown.

The highlighting process follows the general structure:

Source text → Lexer → Token ranges → Highlighter → Styled text

The project uses a common `SyntaxHighlighter` class. It handles shared functionality such as caching, viewport limiting and the styling process.

`KotlinHighlighter` and `MarkdownHighlighter` provide the language-specific span calculation.

5.1 Kotlin Lexer

For Kotlin syntax, the project uses a hand-written `KotlinLexer`.

The lexer performs a single-pass scan of the source text and identifies syntax such as:

- Keywords
- Strings
- Comments
- Annotations
- Numbers
- Function names

The lexer produces ranges within the original source text instead of creating a separate substring for every token. This reduces unnecessary

memory allocation when processing larger files.

Plain identifiers that do not require special highlighting are not emitted as separate tokens. This means the number of generated spans depends more on the amount of syntax that needs styling than simply on the total number of characters in the file.

The lexer also handles Kotlin-specific cases including:

- Nested block comments
- Raw triple-quoted strings
- Annotation use-site targets such as `@file:JvmName`
- Hexadecimal numbers
- Binary numbers
- Exponent forms
- Numeric suffixes
- Function declaration names

Function names are identified by checking whether the previous meaningful token was `fun`.

The lexer also stops an ordinary string at the end of a line. This prevents an unmatched quotation mark from causing the remainder of the file to be treated as a string.

Related component: C5 – Kotlin Syntax Engine.

5.2 Viewport-Based Highlighting

Applying syntax styling to an entire large file after every keystroke would create unnecessary work for Compose.

The implementation therefore separates **analysis** from **styling**.

The complete file is still scanned when the text changes. This is necessary because starting the lexer only from the visible area could produce incorrect results if the visible region begins inside a string, comment or Markdown fenced block.

The results of the scan are cached against the current text, so the complete analysis does not need to be repeated unless the text changes.

Only the spans that intersect the visible editor area are applied to the displayed text.

The visible range is expanded using an 8,000-character quantum with a 4,000-character margin above and below. This prevents small scrolling movements from continuously invalidating the styling window while still keeping highlighted content available around the viewport.

This approach was introduced because styling an entire large file on every edit produced a significant performance cost. In testing, a 4,400-line file generated tens of thousands of spans and caused substantial frame drops. Viewport-limited styling reduced the amount of work performed during normal editing and scrolling.

5.3 Applying the Highlighting

Highlighting is applied using Compose's `OutputTransformation`.

The transformation changes how the source text is displayed but does not modify the actual text stored in the editor state.

Therefore, syntax colours are purely presentational. When the file is saved, the original source text is written to the document without any styling information being stored in it.

6. Markdown Preview

Markdown editing and Markdown preview were implemented as two separate processes because they have different requirements.

The `MarkdownLexer` is responsible for identifying Markdown syntax while the document is being edited. It preserves the original character offsets so that the editor can continue to map positions correctly.

The `MarkdownParser` is responsible for creating the actual preview. Instead of simply colouring the Markdown source, it converts the document into a nested Abstract Syntax Tree (AST).

The parser supports the implemented Markdown structures including:

- Headings
- Bold and italic emphasis
- Strikethrough (~~...~~)
- Ordered lists
- Unordered lists
- Task items
- Blockquotes
- Tables
- Fenced code blocks
- Links
- Images
- Thematic breaks

The parser processes block-level and inline structures separately. Nested structures such as lists inside blockquotes can therefore be represented correctly in the resulting AST.

For emphasis, the parser processes longer delimiter sequences before shorter ones. Code spans are also handled separately so that characters inside backticks are not incorrectly interpreted as Markdown emphasis markers.

The resulting AST is rendered by `MarkdownDocument`, a Jetpack Compose component. Different Markdown structures are mapped to appropriate layouts:

- Headings use larger text styles
- Tables are displayed as aligned rows
- Task items are displayed using checkboxes
- Fenced code blocks use a monospaced layout
- Images are shown as placeholders with alt text and URL (the

- image file itself is not loaded)
- Strikethrough is rendered with a line through the text

Fenced blocks marked as Mermaid are routed to the project's graph renderer when they use the supported flowchart subset (`graph / flowchart` with `TD` or `LR`). Other Mermaid diagram types fall back to a normal code block.

The preview is displayed as an overlay rather than a separate navigation route. This allows the user to close the preview and return to the editor while retaining the existing cursor and scroll position.

Related component: C6 – Markdown Engine.

7. Version Comparison

The comparison feature compares a selected historical version with the **current editor buffer**. Two stored versions cannot be compared with each other.

The project uses `java-diff-utils` version 4.15 for line-based comparison. The library implements the Myers diff algorithm.

The library produces a sparse list of deltas describing the regions that changed. Each library delta is classified as:

- `INSERT`
- `DELETE`
- `CHANGE`
- `EQUAL`

This sparse result is useful for identifying changes but is not directly suitable for the comparison screen. The UI needs every line in order, including unchanged lines, together with independent line numbers on both sides.

Therefore, `DiffEngine.compare()` converts the sparse diff into the line representation used by the comparison UI. The conversion itself is a private `expand()` helper. Library delta types are mapped onto UI types `ADDED`, `REMOVED` and `UNCHANGED`. A `CHANGE` is emitted as removed lines followed by added lines.

7.1 Diff Expansion Approach

The private `expand()` helper processes the deltas in source order and maintains two independent cursors:

- One cursor for the old version
- One cursor for the new version

For an unchanged section, the corresponding lines are emitted on both sides and both cursors advance.

For an insertion:

- A line is added to the new side.
- Only the new cursor advances.
- The old side has no corresponding line number.

For a deletion:

- A line is added to the old side.
- Only the old cursor advances.
- The new side has no corresponding line number.

For a change:

- The removed lines are emitted on the old side.
- The added lines are emitted on the new side.
- The corresponding cursors advance independently.

After processing the main delta list, trailing loops handle any remaining lines at the end of either file.

The resulting `DiffLine` structure uses nullable line numbers to

represent whether a line exists on a particular side. An inserted line has no old line number, while a deleted line has no new line number.

The comparison screen displays a **unified** list rather than two side-by-side panes. Each row shows the old line number, the new line number, a + or - marker, and the line text. A missing number is shown as -. When the reconstructed history version matches the current editor content, the screen shows a “No differences” empty state instead of the line list.

Related component: C9 – Diff Engine.

8. Data and Version Tracking

Version tracking was implemented using a snapshot-and-delta approach.

The purpose of this design is to avoid storing a complete copy of the file for every saved version.

The first version of a file is stored as a complete `SNAPSHOT`.

Every later version is stored as a `DELTA`, containing a unified diff between the previous version and the new version.

For example:

Version	Stored data
V1	Complete file snapshot
V2	Delta from V1 to V2
V3	Delta from V2 to V3

A file therefore has exactly one initial snapshot, followed by delta records.

The deltas are generated using a context radius of zero. Context lines

are normally useful when applying a patch to content that may have changed independently. In this project, however, every delta is applied to the exact previous version from which that delta was generated, so the additional context is not required.

8.1 Version Reconstruction

When an older version is requested, the application reconstructs the version by replaying the version chain.

The process starts from the first snapshot and processes versions in chronological order:

Snapshot → Delta → Delta → Delta → Requested Version

A snapshot replaces the current reconstructed content with its stored full text.

A delta is then applied to the current reconstructed content.

The process stops once the requested version is reached. This means that requesting an earlier version does not require processing versions created after it.

For example, if a file has fifty versions and the user requests version two, versions three through fifty do not need to be processed.

The reconstruction operation is implemented by
`PatchEngine.reconstruct()`.

8.2 Using Reconstruction for Comparison and Rollback

The same reconstruction mechanism is reused by two different features.

For **version comparison**, the selected historical version is

reconstructed and passed to the `DiffEngine` together with the current editor content.

For **rollback**, the selected historical version is reconstructed and then written into the editor buffer. The document on disk is not updated until the user saves.

This avoids implementing separate mechanisms for retrieving historical content.

Therefore, rollback is essentially:

Version Reconstruction → Write Reconstructed Content to Editor

8.3 Creating a New Version

File → Save writes the document through SAF. It does **not** create a `SNAPSHOT` or `DELTA`.

A history version is created only when the user taps **Save Version** on the Version History screen (an optional label may be entered). That action then:

1. Reconstructs the previous version.
2. Compares the current editor content with the reconstructed previous content.
3. Generates a unified diff.
4. Stores the resulting delta as the new version.

This means that later versions store only the changes rather than another complete copy of the file. The storage requirement is therefore mainly related to the amount of content that changes between versions.

Related components: C10 – Incremental Delta / Patch, C11 – Version Control, and C12 – History and Rollback.

9. Rollback Behaviour

Rollback allows the user to restore a previous version of the file.

The process uses the same reconstruction mechanism described in the previous section.

When the user selects a version for rollback:

1. The selected version is reconstructed from the snapshot and required deltas.
2. The reconstructed content is passed back to the editor.
3. The editor replaces its current content with the reconstructed content and **clears the undo/redo history**.

Rollback does not write the reconstructed text to the user's document on disk. The file remains at the last File → Save until the user saves again. After rollback, the editor content differs from the last-saved snapshot, so the unsaved-changes path applies.

During development, a problem was identified where the restored content could be overwritten by the normal file-loading process. When returning from Version History, the editor's loading effect could reload the file from disk and replace the newly restored content.

This was fixed by routing restored content through a pending-rollback channel. The normal loading process checks this state before performing its regular disk reload.

Rollback is a destructive operation because restoring an older version can replace unsaved changes. Therefore, the operation is protected by a confirmation dialog. The dialog identifies the target version and warns the user that unsaved changes will be lost.

Rollback can be accessed from both the version history list and the comparison screen, and both paths use the same confirmation process.

Related component: C12 – History and Rollback.

10. Testing and Verification

The main processing logic was implemented as pure Kotlin where possible, without direct Android dependencies. This allows the lexer, parser, diff, patch and version-management logic to be tested using JVM unit tests without an emulator.

The project contains **98 unit tests across 15 test classes**, and all of these tests pass.

The test coverage includes:

Test class	Main coverage
<code>KotlinLexerTest</code>	Keywords, nested comments, raw strings, annotations and numeric forms
<code>MarkdownLexerTest</code>	Token ranges and offset preservation
<code>MarkdownParserTest</code>	Block/inline parsing, nesting, tables and task items
<code>DiffEngineTest</code>	Delta expansion, line numbering and identical-input cases
<code>PatchEngineTest</code>	Delta creation, application and round-trip correctness
<code>VersionManagerTest</code>	First version as SNAPSHOT and later versions as DELTA
<code>RollbackManagerTest</code>	Reconstruction of an arbitrary version
<code>RecoveryStorageTest</code>	Encoding/decoding, multi-line content and corrupted input

RecoveryManagerTest	Dirty-flag cache, no prompt when clean, discard/clear
PersistenceContractTest	File, version and delta persistence contracts
SettingsRepositoryTest	DataStore settings round-trip
SmartEditorTest	Auto-indent, auto-close brackets and tab-to-spaces
GoToLineTest	Line-offset calculation
RegressionContractTest	Cross-component checks against regressions
ExampleUnitTest	Default JVM template test included in the 98-test count

The tests therefore cover the major logic behind syntax highlighting, Markdown processing, comparison, patching, version creation, reconstruction, rollback and crash recovery.

Android-dependent features were tested separately using an emulator. These included:

- Storage Access Framework file selection
- Crash recovery after a force-stop
- Editor input behaviour
- Rollback through the UI

The complete version-control flow was also tested end-to-end. This included checking the history list, confirming the first version was stored as a snapshot, confirming later versions were stored as deltas, checking the unified diff with dual line numbering, and testing rollback with its confirmation step.

11. Component Summary

The main implementation areas discussed in this report are connected through the following components:

Component	Implementation area
C3	File System / SAF
C5	Kotlin Syntax Engine
C6	Markdown Engine / Preview
C7	Crash Recovery
C8	Database / Persistence
C9	Diff Engine
C10	Incremental Delta / Patch
C11	Version Control
C12	History and Rollback

These components form a dependency chain. File persistence provides the foundation for version storage. The diff engine identifies changes between versions, the patch system stores and applies those changes, version control manages the resulting history, and the history component uses reconstruction for viewing and rollback.

12. Conclusion

Kotlin++ Lite implements its main editing, storage and version-management features through a set of independent but connected components.

The Storage Access Framework is used for the user's actual Kotlin, Markdown and plain-text files, while Room stores file, version and delta information. A separate private recovery file protects unsaved editor content without overwriting the user's saved document.

Syntax highlighting is implemented using hand-written lexers and a shared highlighting system. The analysis is cached and styling is limited to the relevant viewport, while `OutputTransformation` ensures that presentation styling does not become part of the saved source code.

Markdown preview uses a separate parser and AST so that Markdown is rendered as formatted content rather than simply displaying the original Markdown markers.

Version comparison uses `java-diff-utils` and the Myers diff algorithm. `DiffEngine.compare()` converts the sparse library output into a unified line list with independent old and new line numbers, comparing a historical version against the current editor buffer.

Version tracking uses one complete snapshot followed by delta records, created from Version History rather than from File → Save. Historical versions are reconstructed by replaying the required deltas, and the same reconstruction mechanism is reused for comparison and rollback.

Crash recovery uses a length-prefixed private journal to safely preserve unsaved content. Rollback reconstructs the selected historical version into the editor (without writing the document on disk until the user saves), with a confirmation step to protect against accidental loss of unsaved work.

The implementation was verified through 98 JVM unit tests together with emulator-based testing of Android-specific behaviour. These mechanisms together provide the storage, preview, comparison, version tracking, recovery, rollback and syntax highlighting functionality of Kotlin++ Lite